CLASE 21

Métodos Especiales (Dunder Methods)

"Enseñándole a Python cómo tratar tus objetos"

Hoy aprenderán:

- ¿Qué son los métodos dunder? (__nombre__)
- __str__ y __repr__ — Cómo se muestra tu objeto
- __len__ — Darle "longitud" a tu objeto
- __eq__ — Comparar objetos con ==
- Crear clases que se comportan como tipos nativos

¿Qué son los Dunder Methods?

Dunder = Double UNDERscore = __nombre__

Son métodos "mágicos" que Python llama automáticamente:

Cuando haces esto...	Python llama esto:
objeto = MiClase()	objeto.__init__()
print(objeto)	objeto.__str__()
len(objeto)	objeto.__len__()
objeto1 == objeto2	objeto1.__eq__(objeto2)
objeto[0]	objeto.__getitem__(0)
for x in objeto	objeto.__iter__()

Ya conocen __init__ — se llama al crear el objeto. Hoy aprenderemos los demás.

💡 Los dunder methods te permiten "personalizar" cómo Python trata a tus objetos

El problema: print() muestra algo feo

SIN __str__:

```
persona = Persona("Ana", 25)
print(persona)

→ <__main__.Persona object at
0x7f8b...>
```

Tipos nativos:

```
lista = [1, 2, 3]
print(lista)  →  [1, 2, 3]  ✓

print(diccionario) → {'a': 1}  ✓
```

¿Por qué la diferencia?

Listas

Tienen __str__ definido
print() sabe qué mostrar

Diccionarios

Tienen __str__ definido
print() sabe qué mostrar

Tus objetos

NO tienen __str__
print() muestra memoria

💡 Solución: Definir __str__ para controlar qué muestra print()

__str__ — Representación para humanos

__str__ define qué muestra print()

SIN __str__

```
print(persona)
→ <__main__.Persona object at 0x...>
```

CON __str__

```
print(persona)
→ "Ana (25 años)"
```

Reglas de __str__:

- Debe retornar un STRING (siempre)
- Debe ser legible para humanos
- Se usa en: print(), str(), f-strings

Piensa en __str__ como: "¿Cómo quiero que se PRESENTE mi objeto al mundo?"

💡 __str__ es para USUARIOS — debe ser fácil de leer

__repr__ — Representación para devs

__repr__ = "representación" técnica del objeto

__str__ → Para USUARIOS

Bonito, legible

`"Ana (25 años)"`

__repr__ → Para DESARROLLADORES

Técnico, recreable

`Persona('Ana', 25)`

Convención: __repr__ debería mostrar cómo RECREAR el objeto

¿Cuándo se usa __repr__?

- En la consola interactiva (REPL)
- En el debugger
- Cuando no hay __str__ definido (fallback)
- Al imprimir listas de objetos

💡 Si solo defines uno, define __repr__ — Python lo usa como fallback para __str__

__str__ vs __repr__ — Resumen

Aspecto	__str__	__repr__
Audiencia	Usuarios	Desarrolladores
Estilo	Legible y bonito	Técnico y preciso
Ejemplo	"Ana (25 años)"	Persona('Ana', 25)
Se usa en	print(obj), str(obj)	repr(obj), consola
¿Necesario?	Opcional	Recomendado siempre

¿Cuál usar?

→ **¿Tu clase la usan usuarios finales?** Define ambos: __str__ bonito, __repr__ técnico

→ **¿Tu clase la usan solo programadores?** Define __repr__, Python lo usa para todo

💡 Siempre define al menos __repr__

__len__ — Darle longitud a tu objeto

¿Qué pasa si haces len(mi_objeto)?

Sin __len__:

```
carrito = CarritoCompras()
len(carrito)
→ TypeError: object has no len()
```

Con __len__:

```
carrito.agregar("Manzana")
carrito.agregar("Pan")
len(carrito) → 2  ✓
```

⚠️ __len__ debe retornar un ENTERO >= 0 (nunca negativo, nunca float)

Útil para clases que contienen elementos:

🛒 **Carrito**
→ número de productos

🎵 **Playlist**
→ número de canciones

⚽ **Equipo**
→ número de jugadores

💡 Si tu objeto "contiene" cosas, probablemente necesita __len__

🟰 __eq__ — Comparar objetos con ==

Por defecto, == compara si son EL MISMO objeto (misma memoria):

```
p1 = Persona("Ana", 25)
p2 = Persona("Ana", 25)
print(p1 == p2)  →  False

¡Son idénticos pero dice False!
Porque son objetos DIFERENTES en memoria
```

Con __eq__ definimos QUÉ significa "igual":

```
# Con __eq__ que compara nombre y edad
p1 = Persona("Ana", 25)
p2 = Persona("Ana", 25)
print(p1 == p2)  →  True  ✓
Mismo nombre y edad = son iguales
```

💡 __eq__ define el SIGNIFICADO de igualdad para tu clase

__eq__ y colecciones

Con __eq__ definido, puedes usar tu objeto en:

✓ Buscar con "in"

```
if Persona("Ana", 25) in personas:
    print("Encontrada!")
```

✓ Contar ocurrencias

```
personas.count(Persona("Ana", 25))
```

✓ Eliminar de lista

```
personas.remove(Persona("Ana", 25))
```

✓ Verificar en sets

```
(necesita __hash__ también)
Lo veremos en cursos avanzados
```

Sin __eq__: "in" solo encuentra si es el MISMO objeto — count() y remove() no funcionan como esperas

💡 __eq__ hace que tu objeto se comporte como los tipos nativos

Otros métodos de comparación

Además de __eq__ (==), existen:

Método	Operador	Significado
`__eq__`	==	igual
`__ne__`	!=	no igual (auto si hay __eq__)
`__lt__`	<	menor que
`__le__`	<=	menor o igual
`__gt__`	>	mayor que
`__ge__`	>=	mayor o igual

Ejemplo útil: Ordenar personas por edad

Con __lt__ definido: personas.sort() → Ordena de menor a mayor

💡 Si defines __eq__ y __lt__, Python puede ordenar tu clase con sort(), min(), max()

EJERCICIO 1: Clase Cancion

Atributos:

- titulo (string)
- artista (string)
- duracion_segundos (int)

Métodos dunder:

- __str__ → "Titulo - Artista (3:45)"
- __repr__ → "Cancion('Titulo', 'Artista', 225)"
- __eq__ → Iguales si mismo título Y artista
- __lt__ → Ordenar por duración

Pruebas esperadas:

```
print(cancion)
→ "Bohemian Rhapsody - Queen (5:55)"

cancion1 == cancion2
→ True si mismo título y artista

canciones.sort()
→ Ordena por duración
```

Pista: minutos = segundos // 60, resto = segundos % 60
Formato con ceros: f'{segundos:02d}' → pone el 0 si hace falta

Equipos de 2-3 | Tiempo: 12 minutos

BREAK — 10 MINUTOS

Volvemos con más dunder methods útiles
y el ejercicio final

__getitem__ — Acceso con corchetes []

Permite usar objeto[indice] como con listas:

```
playlist = Playlist()
playlist.agregar(cancion1)
playlist.agregar(cancion2)

# Con __getitem__ definido:
print(playlist[0])   → Primera canción
print(playlist[1])   → Segunda canción
print(playlist[-1])  → Última canción
```

También permite:

`Slicing: playlist[0:3]`

Obtener una porción

`for cancion in playlist:`

Iteración automática

💡 Hace que tu objeto se comporte como una lista

__contains__ — El operador "in"

Permite usar "elemento in objeto":

```
# Con __contains__ definido:
if "Bohemian Rhapsody" in playlist:
    print("¡La tenemos!")

if usuario in equipo:
    print("Es parte del equipo")
```

__contains__ recibe el elemento a buscar. Debe retornar True o False.

Sin __contains__:

- Python usa __iter__ o __getitem__
- Funciona pero puede ser lento

Con __contains__:

- Puedes optimizar la búsqueda
- Definir qué significa "estar contenido"

💡 Útil cuando quieres control sobre cómo se busca en tu objeto

Resumen de Dunder Methods Útiles

Representación

`__str__`	print(obj), str(obj)	Para usuarios
`__repr__`	repr(obj), consola	Para devs

Colecciones

`__len__`	len(obj)	Tamaño/longitud
`__getitem__`	obj[i], for x in obj	Acceso por índice
`__contains__`	x in obj	Búsqueda

Comparación

`__eq__`	obj1 == obj2	Igualdad
`__lt__`	obj1 < obj2, sort()	Menor que

💡 Con estos 7 métodos, tu clase se comporta como un tipo nativo de Python

EJERCICIO 2: Clase Inventario completa

Atributos:

- nombre (string) — nombre de la tienda
- productos (lista) — diccionarios {nombre, cantidad, precio}

Métodos normales:

- agregar(nombre, cantidad, precio)
- vender(nombre, cantidad) — reduce stock

Métodos dunder:

- __str__ → "Inventario 'MiTienda' (5 productos)"
- __repr__ → "Inventario('MiTienda')"
- __len__ → Número de tipos de productos
- __getitem__ → Acceder a producto por índice
- __contains__ → Verificar si producto existe

Pruebas esperadas:

```
print(inventario)
→ "Inventario 'TechStore' (3
productos)"

len(inventario) → 3

inventario[0] → Primer producto

"Laptop" in inventario → True

for producto in inventario:
    print(producto)  # funciona
```

Equipos de 2-3 | Tiempo: 18 minutos

Revisemos la solución

[Vamos a VS Code]

✅ **__str__ usa len(self) para mostrar cantidad**
Reutiliza __len__ internamente

✅ **__len__ retorna len(self.productos)**
Número de TIPOS de productos, no unidades totales

✅ **__getitem__ retorna self.productos[indice]**
Permite for loop y acceso con corchetes

✅ **__contains__ busca por nombre (case-insensitive)**
"laptop" encuentra "Laptop"

✅ **agregar() actualiza si ya existe**
No duplica productos

💡 Todos los dunder methods trabajan JUNTOS para crear una clase completa

Resumen — Métodos Especiales (Dunder)

Método	Activa cuando...	Retorna
`__str__`	print(obj), str(obj)	string legible
`__repr__`	repr(obj), debugger	string técnico
`__len__`	len(obj)	int >= 0
`__eq__`	obj1 == obj2	True / False
`__lt__`	obj1 < obj2, sort()	True / False
`__getitem__`	obj[i], for x in obj	elemento
`__contains__`	x in obj	True / False

Filosofía:

- Los dunder methods hacen que tus clases "encajen" en Python
- Tu objeto puede comportarse como una lista, número, string...
- Define solo los que tengan sentido para tu clase

Próxima clase: Proyecto OOP completo + Módulos — "Juntamos todo en un sistema real"